Radar Posture Classification: A Comparative Study of SOTA Architectures

Eloan Tourtelier
CentraleSupélec
eloan.tourtelier@student-cs.fr

Othmane Ouali
CentraleSupélec
othmane.ouali@student-cs.fr

Lucas Cheylan
CentraleSupélec
lucas.cheylan@student-cs.fr

Meriem Mellouki
CentraleSupélec
meriem.mellouki@student-cs.fr

Zaineb Tarhda
CentraleSupélec
zaineb.tarhda@student-cs.fr

CONTENTS

ABSTRACT

In this work, we study and benchmark different deep learning archietcture classifying Human Activity Recognition (HAR) (eg: running, jumping, sitting etc..) for radar data. Point cloud data offer strong potential due to their privacy-preserving and environment-agnostic properties (lighting conditions etc). However, point cloud-based HAR faces challenges like data sparsity, high computation cost, and a lack of large annotated datasets.

The archietcture we study in this paper are PointNet, a skeleton predictor based on a DGCNN, methods using moments caracterisation and simpler machine learning classfieeris to compare as a baseline. We becnhcmarked our models on 4 datsets, all recorded with mmWave radars, and ranging from 2 to 10 classes. Our results highlight which architectural choices are most effective for sparse radar point data and provide practical guidance for future radar-based human state recognition.

1 DATASETS USED

We studied four radar point dataset, respectively radHAR, DGUHA, MARS and a datset we recoreded ourselves, whihc we named Radatatouille. They all comport different classes and differetn features, but all of which have a time dimension, whihc some architectures use and some dont. Each "image" is called a frame, and each frame contains from 4 to 40 radar points, whihc requires some strategy to make that amount fixed. A group of frames is called a windows, and windows coantain a fixed amount of frames. We aim at keeping our windows to last around 2 seconds, whihc is realisitic time for out syteme to predict what pose the user is doing.

1.1 RadHAR

The RadHAR dataset, introduced by Singh *et al.* [6] at the 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems (mmNets, 2019), is one of the most widely used public benchmarks for human activity recognition (HAR) from millimeter-wave (mmWave) radar point clouds. It is also referred to in the literature as the *MMActivity* dataset.

Acquisition setup. The data was collected with a Texas Instruments `IWR1443BOOST` mmWave FMCW radar, a low-cost, commercial off-the-shelf sensor operating in the 76–81 GHz band. The radar was mounted on a tripod at a height of 1.3 m, facing the subject. The sampling rate is approximately 30 Hz, i.e. one frame every ≈ 33 ms.

Activities and subjects. Two participants performed five activities in front of the radar: boxing, jumping, jumping jacks, squats and walking. The dataset is shipped with a fixed

Dataset	Radar used	Classes	Total frames	Features
RadHAR / MMActivity	TI IWR1443BOOST	5	167,632	x, y, z, range, velocity, bearing, intensity
RaDataTouille	TI xWR68xx AOP	2	27,411	x, y, z, SNR, velocity, noise
DGUHA	TI IWR1443BOOST mmWave radar	7	307,200	x, y, z
MARS	TI IWR1443BOOST mmWave radar	10	40,083	x, y, z, Doppler velocity, intensity

train/test split, provided as separate `train/` and `test/` folders in the official repository. We use this split as-is throughout the paper, so that all our experiments remain comparable with prior work using RadHAR. The limited number of subjects (only two) is a critical caveat, however. Even with a clean train/test separation, the model can exploit subject-specific cues—gait rhythm, body size, characteristic arm swing. Near-ceiling accuracy on RadHAR should therefore be read as performance on a *closed-world* two-subject task, not as evidence of generalizable activity recognition.

For every detected point, the following features are provided:

Feature	Description
x, y, z	Cartesian coordinates of the point (m)
r (`range`)	Radial distance from the radar (m)
v (`velocity`)	Doppler radial velocity (m/s)
θ (`bearing`)	Azimuth angle (rad)
I (`intensity`)	Reflected signal intensity

These features are not independent: Cartesian coordinates (x, y, z), range r and bearing θ are linked by the spherical-to-Cartesian relation, so the raw representation is *redundant*. Which subset is fed to a model is therefore a non-trivial design choice. In practice we found the five-dimensional representation (x, y, z, v, I) sufficient, discarding the redundant polar coordinates. The different feature combinations we explored are detailed in Section 2.1.

Effective dataset size. After windowing, the 167,632 frames yield approximately 2,793 labeled samples of 60 frames each. This is the quantity that actually limits classifier capacity: a model trained on this dataset sees roughly 560 samples per class on average, which is modest by deep-learning standards and makes per-subject variability a real concern given that only two subjects contributed data.

Split	Files	Frames
Train	142	128,947
Test	58	38,685
Total	200	167,632

Temporal windowing. Following the original RadHAR protocol, activities are classified over a sliding time window of 2 seconds, which at 30 fps corresponds to a sequence of 60 consecutive frames. Each such window constitutes one labeled sample for the classifier. The resulting object is therefore a *sequence of sets*: 60 unordered, variable-size point sets, collectively carrying a single activity label.

1.2 RaDataTouille

RaDataTouille is the dataset we recorded ourselves with a Texas Instruments `xWR68xx AOP` radar operating at 60 GHz. The AOP (Antenna-on-Package) form factor integrates the antenna array directly on the chip, making the sensor significantly more compact than the tripod-mounted IWR1443BOOST used for RadHAR and MARS. This compactness is directly relevant to the project's target application: a sensor that must be embedded in furniture or worn by a user cannot use a large benchtop board.

The dataset focuses on a binary posture-recognition problem with two labels: `sit` and `stand up`. Although this is a simpler classification problem than the five- or ten-class benchmarks, it is arguably the most deployment-relevant setting in the project: detecting sedentary behavior or signaling fall-risk transitions requires precisely this kind of coarse, robust posture discrimination.

Each detected radar point is represented by six features:

$$(x, y, z, \text{snr}, \text{velocity}, \text{noise}).$$

The split contains 13 training records and 6 test records, totaling 19 recordings, 20,031 training frames and 7,380 test frames (27,411 frames overall). At roughly $15\times$ smaller than RadHAR by frame count, it is a meaningful stress test of whether the studied architectures generalize under limited supervision. The recordings are handled at 10 fps in our tooling, so a 2-second window corresponds to 20 frames.

Split	Records	Frames
Train	13	20,031
Test	6	7,380
Total	19	27,411

1.3 DGUHA

DGUHA, the Dongguk University Human Activity dataset, was introduced by Lee and Kim in the MTGEA paper [3]. It contains synchronized radar point clouds and Kinect recordings, with seven activity labels: running, jumping, sitting down and standing up, both upper-limb extension, falling forward, right-limb extension and left-limb extension.

Each radar point used in this repository carries three features: Cartesian coordinates (x, y, z).

Each clip consists of 400 frames, and each frame is fixed at 25 points. Unlike RadHAR's sliding-window protocol, each clip is a complete, semantically bounded action: the 400-frame window was chosen to span one full execution of the movement. Since each activity recording lasts approximately 20 seconds, DGUHA is about 20 fps, so a 2-second window corresponds to about 40 frames. The train/test split contains

607 training clips and 161 test clips (768 total, 307,200 frames). The sensor is a Texas Instruments `IWR1443BOOST` mmWave radar—the same board used for RadHAR and MARS—paired with an Azure Kinect for ground-truth annotations.

Split	Clips	Frames
Train	607	242,800
Test	161	64,400
Total	768	307,200

1.4 MARS

MARS, for *mmWave-based Assistive Rehabilitation System for Smart Healthcare*, is a rehabilitation-oriented dataset containing synchronized mmWave radar data and Kinect skeleton annotations. It was collected with a Texas Instruments `IWR1443BOOST` mmWave radar and targets ten rehabilitation movements: left upper-limb extension, right upper-limb extension, both upper-limb extension, left front lunge, right front lunge, squat, left side lunge, right side lunge, left-limb extension and right-limb extension.

Each radar point carries five features:

$$(x, y, z, \text{Doppler velocity}, \text{intensity}),$$

where x is the lateral axis, y is the depth axis and z is the vertical axis.

The repository split follows a *subject-based* protocol: recordings from different participants are assigned exclusively to train or test, so the evaluation measures genuine inter-subject generalization rather than interpolation within a single subject's sessions. This is a stricter and more realistic split than a random frame or window partition. The split contains 29 training recordings and 10 test recordings, corresponding to 28,221 training frames and 11,801 test frames. The radar frame duration is 100 ms (10 fps), so a 2-second window corresponds to 20 frames.

Note on data leakage in published results. The original work associated with MARS reported results using a *frame-level* random split rather than a subject-based one. Because consecutive frames from the same recording are highly correlated — a window starting at frame t and one at frame $t+1$ share 59 of their 60 frames — randomly partitioning at the frame level places nearly identical windows in both training and test sets. The model effectively memorizes specific movement executions rather than learning motion patterns that generalize across subjects. Results obtained under this split are substantially inflated and are not reproducible under a subject-isolated evaluation. All our own MARS experiments use the subject-based split described above.

Split	Recordings	Frames
Train	29	28,221
Test	10	11,801
Total	39	40,083

2 SKETCHING + TRANSFORMER

2.1 Preprocessing: Random Fourier Feature sketching

Why preprocessing is needed. A raw radar frame is an *unordered set of points of variable cardinality*. No standard learning architecture — be it an MLP, a CNN, a Transformer or a gradient-boosted tree — can consume such an object directly: they all expect inputs of fixed shape and well-defined structure. A preprocessing step is therefore unavoidable. The original RadHAR baseline addresses this issue through *voxelization*: each 2-second window is discretized into a tensor of shape $60 \times 10 \times 32 \times 32 = 614\,400$ scalar features [6]. This representation is extremely high-dimensional and overwhelmingly sparse, since a typical frame contains only a few tens of points scattered in a grid of 10,240 voxels. The downstream model thus has to invest most of its capacity in processing empty space.

We propose instead to compress each frame into a compact, fixed-size vector via a *random Fourier feature map*, which preserves the information carried by the points while keeping the dimensionality manageable.

Random Fourier feature sketching. For a given choice of d per-point features (see paragraph *Feature selection* below), a frame at time t is a point set $\mathscr{F}_t = \{p_1, \ldots, p_{N_t}\} \subset \mathbb{R}^d$, where N_t varies from frame to frame. We map this set to a fixed-size sketch $s_t \in \mathbb{R}^m$ in two steps.

Step 1 — Per-point random projection. We draw a single random matrix $W \in \mathbb{R}^{m \times d}$ with i.i.d. entries

$$W_{ij} \sim \mathcal{N}(0, \sigma^2),$$

once and for all at the beginning of training, and freeze it. Each point p_i is mapped to

$$\varphi(p_i) = \sin(W p_i) \in \mathbb{R}^m.$$

This is a Fourier-type random feature map (with bias $b = 0$ and using the sine component only), inspired by the random Fourier feature construction of Rahimi and Recht [5]. With $b = 0$ and a single trigonometric component, the inner product $\langle \varphi(p_i), \varphi(p_j) \rangle$ does not strictly approximate a Gaussian kernel; we use it nonetheless as a randomized, non-linear feature map, which proved sufficient for our task.

Step 2 — Aggregation across the frame. Since $\mathscr{F}_t$ is a *set*, the aggregation across its points must be permutation-invariant. We consider two variants:

- **Uniform sketch:** $s_t = \dfrac{1}{N_t} \displaystyle\sum_{i=1}^{N_t} \sin(W p_i)$.
- **Intensity-weighted sketch:** $s_t = \dfrac{1}{\sum_i I_i} \displaystyle\sum_{i=1}^{N_t} I_i \sin(W p_i)$,

where I_i denotes the reflected intensity of point i. The intensity-weighted variant gives more importance to high-confidence reflections, which in our experiments proved beneficial.

The whole window is then represented by a tensor of shape $60 \times m$, obtained by stacking the 60 per-frame sketches.

Feature standardization. Per-point features live on widely different scales: range is in metres ($r \in [0, 10]$ approximately), bearing in radians ($\theta \in [-\pi, \pi]$), velocity in m/s, intensity on an arbitrary scale. Because the random projection $W p_i$ mixes all features linearly, this scale heterogeneity would make the choice of σ ill-defined. We therefore standardize every feature to zero mean and unit variance *before* applying the sketching step. The per-feature mean μ and standard deviation s are computed *on the training set only*, and the same statistics are reused at test time, so that no information leaks from the test set into the preprocessing pipeline.

Feature selection. The six raw features per point $(x, y, z, r, v, \theta, I)$ are redundant: (x, y, z), r and θ are linked by the spherical-to-Cartesian relation. Determining which subset is most informative is therefore an empirical question. We compared four configurations:

Configuration	Per-point features	Aggregation
All features	$(x, y, z, r, v, \theta, I)$	uniform
Cartesian + I	(x, y, z, I)	uniform
Polar + I	(θ, r, I)	uniform
Polar + I (weighted)	(θ, r, I)	I-weighted

The polar configuration (θ, r, I) consistently outperformed the Cartesian variants, which motivated the introduction of the intensity-weighted sketch as a refinement of that configuration. A detailed comparison is reported in the ablation study (Section 2.4).

Sliding-window construction. Each raw recording is segmented into 2-second windows (60 consecutive frames) using a sliding window with a stride of 10 frames. This yields a controlled overlap (50 frames, $\approx 83\%$) between consecutive windows: small enough to keep them distinct from the classifier's point of view, large enough to make full use of the available recordings. The procedure produces a total of 12 150 training windows and 3 565 test windows, in line with the official RadHAR train/test split.

Hyperparameter choices. The preprocessing pipeline introduces several hyperparameters whose impact is non-trivial: the random-projection scale σ, the sketch dimension m, the sliding-window stride, the per-point feature subset, and the aggregation scheme (uniform vs. intensity-weighted). All of them were studied empirically; the corresponding ablation study is reported in Section 2.4. Unless stated otherwise, the rest of the paper uses $\sigma = 1$, $m = 256$, polar features (θ, r, I), intensity-weighted aggregation, and a stride of 10 frames.

Summary. The full preprocessing pipeline can be summarized as

$$\begin{aligned} &\text{recording} \to \text{frames} \to \text{2-s windows} \\ &\qquad \to \text{sketched windows} \to \text{model input.} \end{aligned}$$

The data finally fed to all models compared in this paper is therefore a tensor of shape $(12\,150, 60, 256)$ for training and $(3\,565, 60, 256)$ for testing.

2.2 Transformer encoder

2.3 Training protocol

2.4 Ablation studies

3 POINTNET

PointNet [4] is a fundamental model. It was built to process an unordered set of 3D points with no pixel grid, no voxel ordering. In this project we use two variants. V1 is the original vanilla PointNet classifier, which we validated on the ModelNet40 3-D object benchmark from the original PointNet paper. V2 adds an explicit temporal processing (over the sequence of frames) using an attention mechanism, enabling each point from each frame to "talk" to themsleves across the time windows.

3.1 Vanilla PointNet

PointNet processes each radar frame as an unordered point set. The point cloud is first processed by a TNet, which mutiplies the distributiion by a matrix to rotate/reflect the data, Each point is then processed independently by a stack of shared MLPs that incresease the dimensionality to 1024. This process is repeated twice in the orgingal paper, but this can be increased. A global max-pooling operation then collapses along the point simension into a single 1024-dimensional descriptor. Because max-pooling is invaraint by permuation, it is invariant to the order in which points are presented. This permutation invaraince is critical for every model processing radar points ,becasue detectors returns points in arbitrary order. Finally, a three-layer MLP head maps the max-pooling output to class logits.

A little more about the two transformation networks (T-Nets): The input T-Net predicts a $D \times D$ (D the feature dimension) matrix that is applied to the raw point coordinates, canonicalizing the point cloud orientation before any learned embedding. Both matrices are regularized in the loss with an orthogonality penalty ($\lambda = 0.001$) to keep the learned transformations to a rotation and/or a reflection, making the lenghts, angles, and lvoumes unchanged; without this constraint, the high-dimensional feature T-Net would easily overfit to arbitrary linear maps, and we only want the Tnet to stick to reorient the distribution.

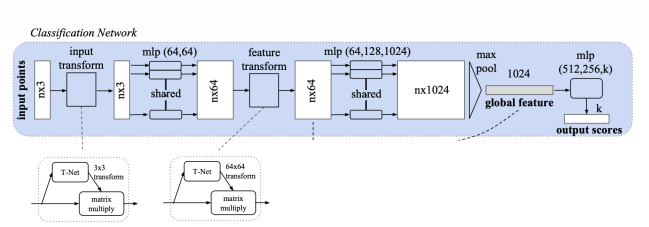

Figure 1: PointNet architecture. The T-Net modules canonicalize point-cloud orientation, while global max pooling produces an order-invariant descriptor.

3.2 Attention PointNet

Within a single frame, points are an unordered set and must be treated symmetrically; across frames, there is a insightful ordering, which carries information. V2 addresses this by stacking a temporal attention mechanism inside the PointNet encoder, right after the TNet rotation.

For the tatention mechanism, we use multi-head self-attention with 8 heads or less (depending if the dimension is large enough for 8 heags). Each frame can attend to all others and weight the time steps most informative for the activity decision. Positional information is injected via *Rotary Position Encoding* (RoPE), which encodes relative temporal distances rather than absolute positions between frames. We add an attention layer after each TNet, and we hence have two blocks TNet –> time Attention –> MLP. The time diemsnion is mean pooled at the very end, after the max pooling (along the N dimension), and right before the final logits generator.

The addition of the time attention mechanism is not always benficial. It seems the longer the time window is (the more frames it contains), the better the accuracy is. Howver, for practical reasons, we keep our windows to last aorund 2 second, which is the time one would expect the radar to classify an input in a commercial scenario.

4 POINTWISE ATTENTION POOLING

This section presents the raw-point attention model studied as a direct classifier for radar activity recognition. It is designed for sparse radar sequences, where each recording is a temporal window of frames and each frame contains an unordered set of radar detections.

4.1 Input representation

For every dataset, the ingestion scripts convert the raw files into the same working format: a sequence of radar frames. Each frame contains points described by their position, velocity and radar response,

$$p_i = (x_i, y_i, z_i, v_i, I_i).$$

The number of detections varies from one frame to another. To obtain fixed-size tensors, we keep at most 32 points per frame and pad shorter frames with empty points. A binary mask is carried through the model so that padded entries do not contribute to the attention weights or to the pooled descriptors.

Training samples are built from windows of 60 consecutive frames with stride 10. The resulting input tensor has shape

$$X \in \mathbb{R}^{B\times T\times P\times F},$$

where $T = 60$ is the number of frames, $P = 32$ is the maximum number of points per frame, and $F = 5$ is the number of point features.

4.2 Model mechanism

The architecture is built around two learned pooling operations. First, a shared pointwise MLP embeds every radar detection independently. Since the same MLP is applied to all points, the model does not rely on an arbitrary ordering inside the point cloud. Then, an intra-frame attention pooling layer learns which detections are useful in each frame. Noisy or padded points can receive a small weight, while points associated with the body or with visible motion can receive a larger one.

After this first pooling stage, every frame is represented by one vector z_t. The sequence $(z_1, \ldots, z_T)$ is passed to a bidirectional GRU, which captures the temporal evolution of the activity. A second attention pooling layer is then applied over the GRU outputs. This gives a single sequence descriptor

$$s = \sum_{t=1}^{T} \alpha_t h_t,$$

where h_t is the GRU state at time t and α_t is the learned temporal attention weight. The final activity class is predicted from s with a small classifier head.

The core idea is therefore to learn two levels of importance: which radar points are informative inside each frame, and which frames are informative inside the temporal window. This makes the model a middle ground between hand-crafted summaries and heavier point-cloud architectures. It still operates on raw detections, but it keeps the pooling operations explicit and relatively lightweight.

4.3 Experimental protocol

The same reference pipeline was run on four cleaned datasets using their native train/test splits. Although the raw formats differ across datasets, the preprocessing step maps all of them to the same frame-based representation with points (x, y, z, v, I). The evaluation protocol is then shared: windows of 60 frames, stride 10, up to 32 points per frame, and 12 training epochs.

4.4 Discussion

Table 1: Results of the pointwise attention pooling model on four radar datasets.

Dataset	Classes	Test windows	Test accuracy	Macro F1
RaDataTouille	2	651	81.41%	0.8118
MARS	10	1,125	74.31%	0.7343
RadHAR	5	3,565	**96.07%**	**0.9616**
DGUHA	7	5,635	92.94%	0.9299

The results in Table 1 show that this mechanism is strong on RadHAR and DGUHA, while MARS and RaDataTouille

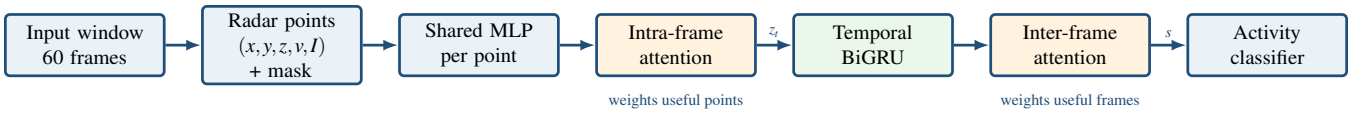

Figure 2: Architecture of the pointwise attention pooling model. The model first learns which radar points to keep within each frame, then learns which frames matter most for the final activity prediction.

remain more difficult. RadHAR is the most favorable setting for the model, with a test accuracy of 96.07%. DGUHA also gives a high score, although it was run on GPU with a larger batch size to keep training time practical. MARS is harder because it contains ten rehabilitation movements with closer visual and kinematic patterns. RaDataTouille is smaller and targets a sit-versus-stand setting, so the model has less data from which to learn robust temporal cues.

This baseline is useful because it keeps the raw radar points while avoiding a large point-cloud backbone. Compared with a moment-based model, it can use the relative importance of individual detections. Compared with heavier architectures, it is easier to train and gives a clear interpretation of where attention is applied: first over points, then over time.

5 MOMENTS

5.1 Motivation

The sketching approach described in Section 2 compresses each frame into a fixed-size vector via a random Fourier feature map. A conceptually simpler alternative is to summarize the empirical distribution of points in a frame by its *statistical moments*: mean, variance, skewness, kurtosis, and range. This representation is fully deterministic, requires no learned or random parameters, and is permutation-invariant by construction.

While statistical moments cannot, in general, uniquely characterize an arbitrary distribution, a classical result in probability theory guarantees that the first $2N$ moments uniquely determine any discrete distribution supported on at most N points [1]. Since each radar frame typically contains fewer than 30 points, a moderate number of moments is therefore *sufficient to characterize the frame exactly*, up to point-set permutation. This property makes the moment representation particularly well-suited to the sparse radar setting, where voxel-based or Fourier-based approaches would introduce unnecessary complexity.

5.2 Preprocessing

Feature selection. Each detected point carries six raw features: x, y, z (Cartesian coordinates), range r, radial velocity v, bearing θ, and intensity I. As noted in Section 1.1, Cartesian coordinates and polar features are geometrically redundant. We found it sufficient to work with a compact five-dimensional per-point representation:

$$p_i = (x_i,\, y_i,\, z_i,\, v_i,\, I_i) \in \mathbb{R}^5.$$

Moment extraction. For a frame $\mathcal{F}_t = \{p_1, \ldots, p_{N_t}\}$, we compute six statistics along each of the $d = 5$ feature dimensions:

Statistic	Definition
Mean	$\mu_k = \frac{1}{N_t}\sum_i p_{ik}$
Std. dev.	$\sigma_k = \sqrt{\frac{1}{N_t}\sum_i (p_{ik}-\mu_k)^2}$
Skewness	$\gamma_{1,k} = \frac{1}{N_t}\sum_i \left(\frac{p_{ik}-\mu_k}{\sigma_k}\right)^3$
Kurtosis	$\gamma_{2,k} = \frac{1}{N_t}\sum_i \left(\frac{p_{ik}-\mu_k}{\sigma_k}\right)^4 - 3$
Minimum	$\min_i p_{ik}$
Maximum	$\max_i p_{ik}$

Concatenating these statistics across all $d = 5$ dimensions yields a fixed-size descriptor

$$s_t = \left[\mu_1, \sigma_1, \gamma_{1,1}, \gamma_{2,1}, \min_1, \max_1, \ldots\right] \in \mathbb{R}^{30},$$

regardless of the number of points N_t in the frame. This descriptor serves as the input to the classifier described below.

Temporal aggregation. In contrast to the sketching pipeline, which operates over a 2-second window of 60 frames and feeds a $(60, m)$ tensor to a sequential model, the moment baseline treats each frame *independently*: a single frame is compressed into one 30-dimensional vector, which is passed directly to an MLP. This amounts to ignoring the temporal structure of the recording. The implications for performance and a discussion of how temporal information could be reintroduced are given in Section 5.4.

Standardization. The 30-dimensional feature vector is standardized to zero mean and unit variance using statistics computed on the training set only, following the same protocol as in Section 2.1.

5.3 Architecture: Multi-Layer Perceptron

Since the 30-dimensional moment vector has no spatial or sequential structure, convolutional or recurrent architectures bring no benefit here. We therefore use a fully-connected MLP with three hidden layers:

The model is trained with early stopping (patience of 10 epochs, validation fraction 15%) to prevent overfitting on the relatively small dataset. No dropout or weight decay was applied, as the model is already severely regularized by the low input dimensionality.

Table 2: MLP architecture used for the moment-based baseline.

Layer	Output dim	Activation	Note
Input	30	—	Standardized moment vector
Dense 1	128	ReLU	
Dense 2	64	ReLU	
Dense 3	32	ReLU	
Output	5	Softmax	One logit per activity class

5.4 Results and Discussion

Table 3 reports per-class precision, recall and F1-score on the RadHAR test set (58 samples, 5 classes).
The model achieves **58.6%** accuracy, well above the random baseline of 20%. Locomotor activities with a clear spatial signature (*walk*: 100% recall; *boxing*: 85% recall) are well captured by the moment representation. By contrast, activities involving repetitive vertical motion confined to a small spatial region (*squats*, *jack*) are poorly recalled (20%), likely because a single-frame moment descriptor cannot encode the periodicity that distinguishes these activities from a static standing posture.

This limitation is a direct consequence of the *temporal blindness* of the current approach. Integrating temporal information — for instance by computing moments over a 2-second sliding window rather than per frame, or by feeding the sequence of per-frame descriptors to an LSTM — is the most natural next step and is expected to close a significant part of the gap with the sequential architectures described in Section 2.

6 SKELETON CLASSIFIER

The SK-DGCNN approach used in the project follows a two-stage strategy: instead of classifying sparse radar points directly, it first maps each radar frame to a structured human skeleton and then classifies the resulting skeleton sequence. The motivation is to make the intermediate representation closer to human motion: a radar point cloud is noisy, sparse and unordered, whereas a skeleton has fixed joints, fixed semantics and a stable graph structure.

6.1 Radar-to-skeleton stage

The first stage is a skeleton-aware DGCNN regressor trained on MARS, where radar frames are synchronized with Kinect skeleton annotations. Each MARS radar frame is cropped or padded to 25 points and represented by five features: x, y, z, Doppler velocity and intensity. A dynamic k-nearest-neighbor graph is recomputed inside the network, and three EdgeConv blocks extract local geometric features. Efficient channel attention is added after each block so that the network can reweight the most informative channels before global pooling.

The output is a centered 19-joint skeleton using the lateral and vertical coordinates (x, z), giving 38 regression targets per frame. We center the skeleton around the `SpineShoulder` joint so that the regressor focuses on body pose rather than absolute position in the radar field of view.

6.2 Skeleton classification stage

Once the radar-to-skeleton checkpoint is trained, it can be used to transform radar recordings into skeleton sequences. In the repository this produces the SKradHAR representation: RadHAR windows are converted from raw radar point clouds into predicted 19-joint skeletons. The activity classifier then operates on this structured sequence rather than on the original point cloud. This separates the perception problem from the classification problem: the first network learns a radar-to-body representation, and the second network learns activity dynamics from body motion.

This design is expected to help when the activity is better explained by joint configuration than by the raw distribution of radar reflections. Its main drawback is error propagation: if the skeleton regressor fails on a frame, the classifier only sees the incorrect skeleton and cannot recover the discarded radar evidence.

7 SIMPLE APPROACHES

8 RESULTS

8.1 Comparison

Table 4 summarizes the comparison grid used for the main experiments.

8.2 Discussion

RadHAR near-ceiling performance and its caveat. PointNet V1 and V2 both reach 96.4% on RadHAR, and DGCNN reaches 95.9%. These numbers look strong but should be interpreted carefully: RadHAR was recorded by only two subjects, so a classifier can achieve high accuracy by recognizing body-shape or gait signatures that are specific to those individuals rather than by learning the activity itself. The practical implication is that RadHAR accuracy figures are systematically optimistic relative to what one would expect on an unseen subject. The two-subject limitation is a ceiling on how much RadHAR results can say about generalization.

Temporal attention provides no lift on RadHAR. V1 and V2 achieve identical test accuracy (96.39%) and near-identical macro F1 (0.9651 vs. 0.9648) on RadHAR. The temporal attention mechanism introduced in V2 adds no measurable benefit here. This is consistent with the per-frame features already being discriminative enough: for activities like walking or boxing, a single-frame point cloud carries a distinctive spatial signature, so modeling how the cloud evolves over time provides diminishing returns on a saturated problem.

Table 3: Per-class performance of the moment-based MLP on RadHAR (test set).

Activity	Precision	Recall	F1-score	Support
Boxing	0.52	0.85	0.65	13
Jack	0.67	0.20	0.31	10
Jump	0.62	0.45	0.53	11
Squats	1.00	0.20	0.33	10
Walk	0.58	1.00	0.74	14
Overall accuracy		**58.6%**		

Vanilla PointNet outperforms the temporal variant on MARS. On the harder, subject-isolated MARS benchmark, V1 (89.3%, macro F1 0.786) actually outperforms V2 (86.1%, macro F1 0.761). This reversal is counterintuitive and suggests a training-efficiency issue: V2's additional attention parameters require more data to optimize reliably, and the MARS training set (3 subjects, 383 windows) may simply not be large enough to take advantage of temporal modeling. It also highlights that architectural complexity is not always beneficial in low-data regimes.

Edge features help on larger action sets. On DGUHA (7 classes), the ranking is DGCNN (95.0%) $>$ PointNet V2 (94.4%) $>$ PointNet V1 (92.6%). The advantage of DGCNN's edge-convolution operators grows with the number of classes, consistent with the intuition that fine-grained local geometry — the relative position and velocity of neighboring points — matters more when distinctions between actions are subtle.

The Moments MLP gap quantifies the value of temporal context. The moment-based MLP reaches only 58.6% on RadHAR, a gap of $\sim$38 percentage points below PointNet V1. Since the MLP is temporally blind (it classifies one frame at a time), this gap directly quantifies how much information is carried by the temporal dynamics of the point cloud. Activities that are spatially ambiguous in a single frame—squats and jumping jacks are both vertical, periodic motions with a small spatial footprint—require temporal context to be disambiguable. This is the single clearest argument in the results for using sequence-aware architectures rather than frame-independent baselines.

Table 4: Global comparison table for the studied datasets and architectures.

Dataset	Architecture	Params	Epochs	Test acc.	Test macro F1
RadHAR	PointNet V1	3,341,909	40	0.9639	0.9651
RadHAR	PointNet V2	3,358,357	40	0.9639	0.9648
RadHAR	DGCNN	–	40	0.9592	0.9608
RadHAR	SK-DGCNN	1,081,093	200	0.8877	0.8879
RadHAR	Moments MLP	14,469	–	0.5860	0.5120
RadHAR	Sketching Transformer	–	–	–	–
RaDataTouille	PointNet V1	3,345,766	–	–	–
RaDataTouille	PointNet V2	3,362,294	–	0.8487	0.8478
RaDataTouille	DGCNN	–	–	–	–
RaDataTouille	SK-DGCNN	–	–	–	–
RaDataTouille	Moments MLP	–	–	–	–
RaDataTouille	Sketching Transformer	–	–	–	–
DGUHA	PointNet V1	3,341,008	–	0.9255	0.9263
DGUHA	PointNet V2	3,357,428	–	0.9441	0.9449
DGUHA	DGCNN	–	–	0.9503	0.9497
DGUHA	SK-DGCNN	1,935,886	–	–	–
DGUHA	Moments MLP	–	–	–	–
DGUHA	Sketching Transformer	–	–	–	–
MARS	PointNet V1	3,346,915	–	0.8934	0.7862
MARS	PointNet V2	3,363,399	–	0.8607	0.7609
MARS	DGCNN	–	–	–	–
MARS	SK-DGCNN	1,933,454	–	–	–
MARS	Moments MLP	–	–	–	–
MARS	Sketching Transformer	–	–	–	–

9 CONCLUSION

A APPENDIX

A.1 PointNet V1 architecture and hyperparameters

PointNet V1 is the vanilla direct point-cloud classifier used in `src/pointnetV1/`. For RadHAR, one training sample has shape $(60, 32, 4)$: 60 frames, 32 points per frame, and the four features x, y, z, velocity. The input windows use a stride of 60 frames. Features are standardized with training-set statistics and clipped at ± 4. Training uses batch size 16, dropout 0.5, Adam with base learning rate 10^{-3}, momentum parameter 0.9, five warmup epochs, label smoothing 0.2, T-Net orthogonality regularization coefficient 10^{-3}, and gradient accumulation 1.

Table 5: PointNet V1 module layout.

Module	Layers	Output
T-Net block 1	$4 \to 64 \to 128 \to 1024 \to 512 \to 256 \to 4^2$	4×4 transform
Encoder block 1	shared MLP $4 \to 64 \to 64$	$(60, 32, 64)$
T-Net block 2	$64 \to 64 \to 128 \to 1024 \to 512 \to 256 \to 64^2$	64×64 transform
Encoder block 2	shared MLP $64 \to 64 \to 128 \to 1024$	$(60, 32, 1024)$
Pooling	max over points, mean over frames	1024
Classifier	$1024 \to 512 \to 5$ with dropout	5 logits

A.2 PointNet V2 architecture and hyperparameters

PointNet V2 uses the same data shape, preprocessing, classifier head and training hyperparameters as V1, but adds temporal attention inside each encoder block before the shared MLP. The attention is applied to each point trajectory across the 60-frame window. With four input features the implementation uses two heads in the first block; with 64 features it uses eight heads in the second block. Queries and keys receive rotary positional embeddings so that attention is aware of relative frame position.

Table 6: PointNet V2 module layout.

Module	Layers	Output
Attention block 1	T-Net $4 \to 4^2$, MHA 2 heads, MLP $4 \to 64 \to 64$	$(60, 32, 64)$
Attention block 2	T-Net $64 \to 64^2$, MHA 8 heads, MLP $64 \to 64 \to 128 \to 1024$	$(60, 32, 1024)$
Pooling	max over points, mean over frames	1024
Classifier	$1024 \to 512 \to 5$ with dropout 0.5	5 logits

A.3 SK-DGCNN architecture and hyperparameters

The SK-DGCNN skeleton regressor is implemented in `src/SK-dgcnn/MARS_skdgcnn.ipynb`. Each sample is one MARS radar frame with shape $(25, 5)$, using $x, y, z,$ velocity, intensity. The model predicts 19 centered skeleton joints with two coordinates per joint, therefore 38 scalar outputs. Training uses batch size 16, test batch size 64, 150 epochs, Adam with learning rate 10^{-3} and momentum parameter 0.5, multi-step decay at epochs 15 and 20 with factor 0.1, dropout 0.4, $k = 20$ neighbors, embedding dimension 512, ECA kernel size 3, and a validation fraction of 0.2. The loss is a sequential multitask combination of MSE and KL-divergence over the z then x skeleton axes, changing phase every 75 epochs.

Table 7: SK-DGCNN skeleton regressor layout.

Module	Layers	Output
EdgeConv 1	graph feature 10, Conv2d $10 \to 64$, BN, LeakyReLU, ECA	$(64, 25)$
EdgeConv 2	graph feature 128, Conv2d $128 \to 64$, BN, LeakyReLU, ECA	$(64, 25)$
EdgeConv 3	graph feature 128, Conv2d $128 \to 128$, BN, LeakyReLU, ECA	$(128, 25)$
Embedding	concat 256, Conv1d $256 \to 512$, BN, LeakyReLU, ECA	$(512, 25)$
Pooling	adaptive max pool + adaptive avg pool	1024
Joint trunk	Linear $1024 \to 512 \to 19 \cdot 128$ with dropout	$(19, 128)$
Heads	Linear $128 \to 1$ for x, Linear $128 \to 1$ for z	38 values

REFERENCES

[1] N. I. Akhiezer. *The Classical Moment Problem and Some Related Questions in Analysis*. Oliver and Boyd, Edinburgh and London, 1965.

[2] Sizhe An and Umit Y. Ogras. Mars: mmwave-based assistive rehabilitation system for smart healthcare. *ACM Transactions on Embedded Computing Systems*, 20(5s):72:1–72:22, 2021.

[3] Gawon Lee and Jihie Kim. Mtgea: A multimodal two-stream gnn framework for efficient point cloud and skeleton data alignment. *Sensors*, 23(5):2787, 2023.

[4] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 77–85, 2017.

[5] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, volume 20, pages 1177–1184, 2007.

[6] Akash Deep Singh, Sandeep Singh Sandha, Luis Garcia, and Mani Srivastava. Radhar: Human activity recognition from point clouds generated through a millimeter-wave radar. In *Proceedings of the 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems*, pages 51–56, New York, NY, USA, October 2019. Association for Computing Machinery. mmNets '19, co-located with MobiCom '19: The 25th Annual International Conference on Mobile Computing and Networking.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[8] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics*, 38(5):1–12, 2019.

[9] Zihan Zhang, Aman Anand, and Farhana Zulkernine. Sk-dgcnn: Human activity recognition from point cloud data with skeleton transformation. *Machine Learning with Applications*, 23:100847, 2026.